Design-Weighted Latent Class Analysis of Survey Data: An Adaptable Analysis Script

Openness to government negotiations as a worked example

Kevin

2026-06-16

1 Purpose and how to use this script

This is a complete, adaptable analysis for finding latent classes (hidden respondent types) in categorical survey items collected under a complex sampling design, estimating the class structure so that it describes the **population** rather than the achieved sample, attaching design-based confidence intervals to every estimated probability, writing class membership back to the respondents, and profiling that membership on demographics.

The worked example identifies types of respondents by their **openness to their government negotiating with a neighboring country**, from a multistage sample of about 1,200 respondents with base weights. Everything that is specific to a survey lives in one configuration block (Section 3). To apply this to your own data, edit that block and nothing else.

The script makes four methodological commitments, each explained where it occurs:

1. **The design is respected during estimation**, not only afterward. Class prevalences and item-response probabilities are estimated by *pseudo-maximum likelihood* with the base weights, which is the analogue of declaring `svyset psu [pweight = w], strata(stratum)` in Stata.
2. **`poLCA` is used as an unweighted benchmark**, because it cannot use design weights, and the gap between it and the weighted fit measures how informative the design is for this typology. A built-in equivalence check confirms the weighted estimator reproduces `poLCA` exactly when all weights are set to one.
3. **Uncertainty is design-based.** Confidence intervals on every conditional and inclusion probability come from the stratified jackknife (JKn) using replicate weights, which respects the stratification and clustering of a multistage design.

4. **"Don't know" is treated as a substantive response category**, not as missing data, on the argument that for an attitudinal item like openness to negotiations, declining to take a side is itself a position. The script notes where to change this if your items call for it.

The code uses the tidyverse, contains no `for` or `while` loops (iteration is done with `purrr` and matrix algebra), and is written to fail loudly and informatively rather than silently on edge cases.

2 Setup

```
library(tidyverse)     # data manipulation and plotting
library(matrixStats)   # numerically stable row-wise log-sum-exp and cumulative sums
library(survey)        # complex-design objects, replicate weights, svyglm
library(poLCA)         # unweighted benchmark latent class model
library(clue)          # solve_LSAP: optimal label alignment across fits

set.seed(2026)
`%||%` <- rlang::`%||%`  # null-coalescing helper used in the EM

# poLCA attaches MASS, whose select() masks dplyr::select(). We therefore
# qualify dplyr::select() everywhere. This is a deliberate, defensive choice.
```

3 Configuration: the only block you edit to reuse this script

Everything design-specific is gathered here. The worked example below the block generates synthetic data that matches this configuration; when you supply your own data, delete the simulation (Section 5) and point `cfg$data` at your data frame.

```
cfg <- list(
  # --- columns in YOUR data ------------------------------------------------
  items   = paste0("q", 1:8),   # the categorical indicator variables for the LCA
  aux     = c("age_grp", "education", "region"),  # demographics for profiling
  strata  = "stratum",          # stratum identifier (design)
  psu     = "psu",              # primary sampling unit / cluster identifier (design)
  weight  = "w",                # base weight (design)

  # --- analysis choices ----------------------------------------------------
  K_range = 2:6,                # candidate numbers of latent classes to compare
```

```
  n_starts = 20L,                # random starts per fit (guards against local maxima)
  dk_as_category = TRUE,         # TRUE: "Don't know" is its own response level
                                 # FALSE: "Don't know" is treated as missing (see Section 8)
  target_class = NA_integer_,    # class to profile; NA = the most prevalent class

  # --- data ------------------------------------------------------------------
  data = NULL                    # set to your data frame; left NULL to use the simulation
)
```

A note on the **`items` coding requirement**, which the script enforces: each indicator must be a small set of consecutive integer codes starting at 1 (for example 1,2,3,4 for a four-point item, with "Don't know" coded as its own level such as 5 when `dk_as_category = TRUE`). The helper in Section 6 checks this and stops with a clear message if an item is mis-coded, because the latent class likelihood indexes response-probability matrices by these integers.

```
# A clean, colorblind-safe plotting theme reused by every figure.
theme_lca <- function(base_size = 11) {
  theme_minimal(base_size = base_size) +
    theme(panel.grid.minor = element_blank(),
          panel.grid.major.x = element_blank(),
          strip.text = element_text(face = "bold", size = rel(0.85)),
          plot.title = element_blank(),
          legend.position = "bottom",
          legend.title = element_text(face = "bold", size = rel(0.85)),
          plot.caption = element_text(hjust = 0, size = rel(0.78), color = "grey30"))
}
class_pal <- function(K) setNames(viridisLite::viridis(K, begin = 0.08, end = 0.85),
                                  paste("Class", seq_len(K)))
wu_pal <- setNames(viridisLite::viridis(2, begin = 0.2, end = 0.75),
                   c("Weighted (population)", "Unweighted (poLCA)"))
```

4 The latent class model

A latent class model supposes that each respondent belongs to one of K unobserved classes, and that **within a class the items are statistically independent** (the local-independence assumption). Writing π_k for the share of the population in class k, and ρ_{jck} for the probability that a member of class k gives response c to item j, the probability of observing respondent i's full pattern of answers is a mixture over the classes:

$$P(\mathbf{y}_i) \;=\; \sum_{k=1}^{K} \pi_k \prod_{j=1}^{J} \rho_{j\, y_{ij}\, k}, \qquad \text{with } \sum_{c=1}^{C_j} \rho_{jck} = 1 \text{ for every item } j \text{ and class } k.$$

The π_k are the **inclusion (class-size) probabilities** and the ρ_{jck} are the **conditional (item-response) probabilities**; together they are the *measurement model*. Each item is treated as an unordered (nominal) categorical variable, which is exactly what `poLCA` assumes, so the benchmark in Section 11 compares like with like. Treating "Don't know" as a category simply adds one response level c to the items where it appears.

4.1 Why ordinary `poLCA` is not enough

`poLCA` maximizes the *unweighted* likelihood, so it estimates the class structure of the **achieved sample**. Under a complex design with unequal selection probabilities, the sample over- or under-represents parts of the population, and the unweighted estimates inherit that distortion. To describe the **population**, each respondent's contribution to the likelihood must be weighted by their base weight w_i, giving the *design-weighted pseudo-log-likelihood*

$$\ell_w(\theta) \;=\; \sum_i w_i \, \log\Big(\sum_{k=1}^{K} \pi_k \prod_j \rho_{j\, y_{ij}\, k} \Big).$$

Maximizing ℓ_w yields *pseudo-maximum-likelihood* estimates of the finite-population parameters. This is the standard design-based device, and it is what we implement below, because `poLCA` has no argument for weights.

5 Worked-example data (delete this section to use your own data)

This section simulates a multistage sample that matches the configuration: 9 strata, about 20 primary sampling units in total, roughly 1,200 respondents, base weights that vary across strata, and a design that is **informative** because the more heavily weighted strata are also the ones least open to negotiation. Eight items measure openness on four-point scales, with a fifth "Don't know" level present on every item for 5 to 10 percent of respondents. Because the data are generated from known parameters, Sections 7 and 9 can show how closely the weighted estimator recovers the truth.

The simulation itself avoids loops: respondents are laid out with `tidyr::expand_grid`, and class draws and item draws are vectorized through inverse-CDF samplers.

```r
# ---- generative truth ------------------------------------------------------
K_true <- 3L
J      <- 8L
n_resp_levels <- 4L                 # substantive responses 1..4
dk_code <- n_resp_levels + 1L        # "Don't know" coded as 5

# Class-conditional response profiles for the 4 substantive levels:
# Class 1 "Open" leans high, Class 2 "Ambivalent" central, Class 3 "Closed" leans low.
prof <- list(
  open       = c(0.08, 0.17, 0.35, 0.40),
  ambivalent = c(0.20, 0.32, 0.30, 0.18),
  closed     = c(0.45, 0.30, 0.17, 0.08)
)
true_rho4 <- list(open = prof$open, ambivalent = prof$ambivalent, closed = prof$closed)

# ---- multistage design -----------------------------------------------------
# 9 strata; PSUs per stratum chosen to total ~20; 60 respondents per PSU.
psu_per_stratum <- c(2L, 2L, 2L, 2L, 2L, 2L, 2L, 3L, 3L)  # sums to 20 PSUs
resp_per_psu    <- 60L
H <- length(psu_per_stratum)

# Stratum-level openness mix (alpha) and base weight, made to covary => informative.
grade   <- seq(0, 1, length.out = H)
alpha_h <- cbind(open       = 0.20 + 0.45 * grade,
                 ambivalent = 0.30,
                 closed     = 0.50 - 0.45 * grade)
alpha_h <- alpha_h / rowSums(alpha_h)
w_stratum <- round(seq(2.2, 0.6, length.out = H), 2)   # open strata sampled more (lower weig

# Build the respondent frame without loops.
frame <- tibble(stratum = rep(seq_len(H), times = psu_per_stratum)) |>
  mutate(psu_in_stratum = sequence(psu_per_stratum)) |>
  mutate(psu = row_number()) |>                                    # unique PSU id
  tidyr::uncount(resp_per_psu, .id = "within") |>
  mutate(id = row_number(),
         w  = w_stratum[stratum])

n <- nrow(frame)

# ---- draw latent class for each respondent (vectorized inverse-CDF) ----------
draw_rows <- function(P) {
  cum <- matrixStats::rowCumsums(P)
```

```
  u   <- runif(nrow(P))
  pmin(rowSums(cum < u) + 1L, ncol(P))
}
true_class <- draw_rows(alpha_h[frame$stratum, , drop = FALSE])

# ---- draw item responses given class, then overlay "Don't know" --------------
rho_by_class <- list(true_rho4$open, true_rho4$ambivalent, true_rho4$closed)

draw_one_item <- function(seed_offset) {
  set.seed(900 + seed_offset)
  P <- do.call(rbind, rho_by_class[true_class])   # n x 4 response probs per respondent
  resp <- draw_rows(P)
  # Don't-know overlay: 5-10% per item, mildly higher among the ambivalent class.
  dk_p <- 0.05 + 0.03 * (true_class == 2)
  is_dk <- rbinom(length(resp), 1, dk_p) == 1
  resp[is_dk] <- dk_code
  resp
}
item_mat <- map(seq_len(J), draw_one_item) |> set_names(paste0("q", seq_len(J)))

sim <- frame |>
  bind_cols(as_tibble(item_mat)) |>
  mutate(true_class = true_class,
         age_grp   = factor(sample(c("18-34","35-54","55+"), n, TRUE)),
         education = factor(sample(c("HS or less","Some college","BA+"), n, TRUE)),
         # region carries a mild signal so a profiling covariate is non-null
         region    = factor(if_else(runif(n) < plogis(-0.3 + 0.6*(true_class==1)),
                                    "Coastal", "Interior")))

# Population truth for later comparison.
pop_pi_true <- as.numeric(prop.table(xtabs(w ~ true_class, data = sim)))

# Hand the simulated data to the pipeline.
cfg$data <- sim
```

6 Preparing and validating the data

Before any modeling, the script standardizes the indicators and checks the design columns, stopping with an explicit message if anything is mis-coded. This is the robustness layer: it converts silent downstream errors into clear, early ones.

The key preparation step concerns "Don't know." When `cfg$dk_as_category = TRUE`, the DK code is left in place as an ordinary response level, so each item's number of categories simply includes it. When `FALSE`, DK values are set to `NA`; the latent class likelihood then uses only each respondent's *answered* items, because under local independence a missing item drops out of the product cleanly, so every respondent is still scored without imputation. The mathematics of that partial-pattern scoring is

$$p_{ik} = \frac{\pi_k \prod_{j \in \text{answered}_i} \rho_{j\, y_{ij}\, k}}{\sum_m \pi_m \prod_{j \in \text{answered}_i} \rho_{j\, y_{ij}\, m}},$$

with the product taken only over items respondent i actually answered.

```
# Pull the working data and confirm required columns exist.
dat <- cfg$data
stopifnot(is.data.frame(dat))
required_cols <- c(cfg$items, cfg$aux, cfg$strata, cfg$psu, cfg$weight)
missing_cols  <- setdiff(required_cols, names(dat))
if (length(missing_cols) > 0)
  stop("These configured columns are not in the data: ", paste(missing_cols, collapse = ", ")

# Handle "Don't know" according to the configuration.
# Convention for this script: the DK level is the largest integer code on an item.
dat_prepared <- dat |>
  mutate(across(all_of(cfg$items), as.integer))

if (!cfg$dk_as_category) {
  # Treat the top code on each item as DK -> set to NA (partial-pattern scoring handles it).
  dat_prepared <- dat_prepared |>
    mutate(across(all_of(cfg$items), ~ if_else(.x == max(.x, na.rm = TRUE), NA_integer_, .x))
}

# Each item must be consecutive integers starting at 1 (ignoring NA). Check and report.
recode_consecutive <- function(x) as.integer(factor(x, levels = sort(unique(x[!is.na(x)]))))
dat_prepared <- dat_prepared |> mutate(across(all_of(cfg$items), recode_consecutive))

cats <- map_int(cfg$items, ~ max(dat_prepared[[.x]], na.rm = TRUE))
names(cats) <- cfg$items
J_items <- length(cfg$items)

# Report the category counts so the analyst can confirm DK handling took effect.
tibble(item = cfg$items, categories = cats) |> knitr::kable(align = "lr")
```

item	categories
q1	5
q2	5
q3	5
q4	5
q5	5
q6	5
q7	5
q8	5

7 The weighted EM estimator

The script estimates the measurement model with a weighted Expectation-Maximization (EM) algorithm. EM alternates two steps until the weighted log-likelihood stops improving.

The **E-step** computes each respondent's posterior probability of belonging to each class, given the current parameters:

$$p_{ik} \;=\; \frac{\pi_k \prod_j \rho_{j\, y_{ij}\, k}}{\sum_m \pi_m \prod_j \rho_{j\, y_{ij}\, m}}.$$

The **M-step** updates the parameters as *weighted* proportions, which is the only place the base weights enter:

$$\hat{\pi}_k \;=\; \frac{\sum_i w_i\, p_{ik}}{\sum_i w_i}, \qquad \hat{\rho}_{jck} \;=\; \frac{\sum_i w_i\, p_{ik}\, \mathbf{1}(y_{ij} = c)}{\sum_i w_i\, p_{ik}}.$$

Because the objective being maximized is the weighted *pseudo*-log-likelihood rather than an ordinary likelihood, these are *pseudo*-maximum-likelihood estimates. They are consistent for the population parameters, but, as Section 10 explains, their standard errors cannot be read off the curvature of the objective in the usual way and must instead come from replication.

Two implementation choices make this robust and loop-free. First, the E-step is computed on the log scale and normalized with the **log-sum-exp** identity, which prevents numerical underflow when many small probabilities are multiplied. Second, the iteration is expressed as a `purrr::reduce` *fold* over iteration indices rather than a `while` loop: each step returns the updated state, and the fold stops improving once converged. Missing items (when DK is treated as missing) are handled by skipping them in the per-item product, so partial response patterns contribute whatever information they carry.

```r
# Random starting values: a random class distribution and random response-prob matrices.
rand_init <- function(cats, K) {
  list(pi  = { x <- runif(K); x / sum(x) },
       rho = map(cats, function(Cj) {
         m <- matrix(runif(Cj * K) + 0.1, Cj, K); sweep(m, 2, colSums(m), "/")
       }))
}

# One full EM run, written as a fold (no loops). Y is a list of integer item vectors
# (NA allowed); OH is a list of one-hot indicator matrices with NA rows zeroed.
em_run <- function(Y, OH, cats, w, K, init = NULL, maxit = 800L, tol = 1e-8) {
  nn <- length(Y[[1]])
  st0 <- c(init %||% rand_init(cats, K),
           list(post = NULL, ll = -Inf, iter = 0L, done = FALSE))

  step <- function(state, .iter) {
    if (isTRUE(state$done)) return(state)
    # E-step on the log scale; NA responses contribute 0 to the log-product.
    log_terms <- map2(state$rho, Y, function(rho_j, y) {
      lp <- log(rho_j)[y, , drop = FALSE]      # nn x K; rows with NA y become NA
      lp[is.na(lp)] <- 0                       # missing item drops out of the product
      lp
    })
    logdens <- reduce(log_terms, `+`) + matrix(log(state$pi), nn, K, byrow = TRUE)
    lse  <- matrixStats::rowLogSumExps(logdens)
    post <- exp(logdens - lse)
    ll   <- sum(w * lse)
    # M-step: weighted proportions. crossprod(OH_j, w*post) sums weighted responsibilities
    # over respondents who gave each category; columns renormalized to valid probabilities.
    wp  <- w * post
    den <- colSums(wp)
    rho_n <- map(OH, function(oh) {
      num <- pmax(crossprod(oh, wp), 1e-12)
      sweep(num, 2, colSums(num), "/")
    })
    list(pi = den / sum(den), rho = rho_n, post = post, ll = ll,
         iter = state$iter + 1L,
         done = abs(ll - state$ll) < tol * (abs(state$ll) + 1))
  }
  out <- reduce(seq_len(maxit), step, .init = st0)
  out$converged <- out$done
  out
```

```r
}

# Build integer item vectors and one-hot matrices (NA rows zeroed so they add nothing).
make_inputs <- function(df, items, cats) {
  Y  <- map(items, ~ df[[.x]])
  OH <- map2(items, cats, function(it, Cj) {
    oh <- outer(df[[it]], seq_len(Cj), `==`) + 0
    oh[is.na(oh)] <- 0
    oh
  })
  list(Y = Y, OH = OH)
}

# Class labels are arbitrary; align any fit to a reference by matching response profiles
# with the Hungarian algorithm, so classes are comparable across starts, fits, and replicates
profiles_of <- function(rho) do.call(cbind, map(rho, t))   # K x sum(Cj)
align_to <- function(fit, ref) {
  K <- length(fit$pi)
  Pf <- profiles_of(fit$rho); Pr <- profiles_of(ref$rho)
  cost <- outer(seq_len(K), seq_len(K),
                Vectorize(function(a, b) sum((Pf[a, ] - Pr[b, ])^2)))
  asg <- clue::solve_LSAP(cost)
  inv <- integer(K); inv[as.integer(asg)] <- seq_len(K)
  list(pi = fit$pi[inv],
       rho = map(fit$rho, ~ .x[, inv, drop = FALSE]),
       post = if (!is.null(fit$post)) fit$post[, inv, drop = FALSE] else NULL,
       ll = fit$ll, converged = fit$converged %||% NA)
}

# Fit at a given K from many random starts; keep the best weighted log-likelihood.
fit_lca <- function(df, w, cats, items, K, starts, ref = NULL) {
  inp <- make_inputs(df, items, cats)
  cands <- map(seq_len(starts), function(s) {
    set.seed(1000 + s + 17 * K)
    em_run(inp$Y, inp$OH, cats, w, K)
  })
  best <- cands[[which.max(map_dbl(cands, "ll"))]]
  if (!is.null(ref)) best <- align_to(best, ref)
  best
}
```

8 Choosing the number of classes

The number of classes K is chosen by comparing models on three quantities. **BIC** and **AIC** trade fit against complexity, using the parameter count $\mathrm{df} = (K-1) + K\sum_j (C_j - 1)$; lower is better, and BIC is the more reliable guide for latent class enumeration because AIC tends to favor too many classes. **Relative entropy** measures how cleanly respondents separate into classes,

$$E \;=\; 1 - \frac{-\sum_i \sum_k p_{ik} \ln p_{ik}}{n \ln K},$$

ranging from 0 (no separation) to 1 (perfect separation); it describes *usefulness of the classification*, not fit, so it is read alongside the information criteria rather than as a selection rule on its own.

All enumeration here uses the **weighted** fit, so the chosen K is the one that best describes the population. The script reports all three quantities across the candidate range and selects the K that minimizes the weighted BIC.

```
df_k <- function(K, cats) (K - 1) + K * sum(cats - 1)

entropy_R2 <- function(post, K) {
  if (K == 1) return(NA_real_)
  1 - (-sum(post * log(pmax(post, 1e-12)))) / (nrow(post) * log(K))
}

w_vec <- dat_prepared[[cfg$weight]]

enum <- map_dfr(cfg$K_range, function(K) {
  fit <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K, cfg$n_starts)
  tibble(K = K,
         logLik = fit$ll,
         df = df_k(K, cats),
         AIC = -2 * fit$ll + 2 * df_k(K, cats),
         BIC = -2 * fit$ll + df_k(K, cats) * log(nrow(dat_prepared)),
         entropy = entropy_R2(fit$post, K),
         converged = fit$converged)
})

K_star <- enum |> slice_min(BIC, n = 1) |> pull(K) |> first()

enum |>
  mutate(across(c(logLik, AIC, BIC), ~ round(.x, 1)),
```

```
         entropy = round(entropy, 3)) |>
  knitr::kable(align = "r",
               caption = "Weighted fit statistics by number of classes. Selection is by minim
```

Table 2: Weighted fit statistics by number of classes. Selection is by minimum BIC.

K	logLik	df	AIC	BIC	entropy	converged
2	-18989.1	65	38108.2	38439.0	0.655	TRUE
3	-18931.4	98	38058.7	38557.5	0.744	TRUE
4	-18873.5	131	38009.1	38675.9	0.748	TRUE
5	-18821.9	164	37971.9	38806.7	0.787	TRUE
6	-18779.3	197	37952.6	38955.4	0.780	TRUE

The selected solution has **2 classes**. The figure shows BIC and entropy across the candidate range; BIC is read for selection and entropy for separation.

```
enum |>
  dplyr::select(K, BIC, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "Weighted BIC", entropy = "Relative entropy"))
  ggplot(aes(K, value)) +
  geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
  geom_point(size = 2.4, color = viridisLite::viridis(1, begin = 0.4)) +
  facet_wrap(~ criterion, scales = "free_y") +
  scale_x_continuous(breaks = cfg$K_range) +
  labs(x = "Number of latent classes (K)", y = NULL) +
  theme_lca()
```

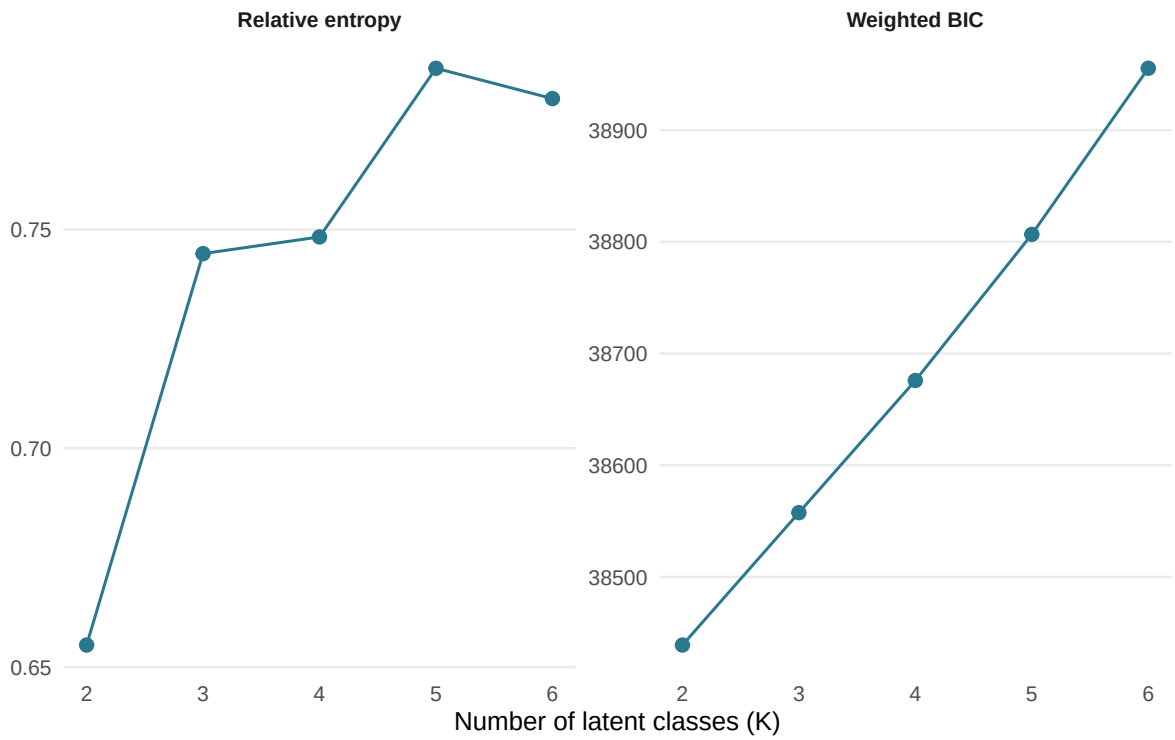

Figure 1: Weighted BIC and relative entropy across candidate numbers of classes. The chosen K minimizes BIC; entropy describes how cleanly respondents are classified.

9 The population measurement model

At the selected K, the script fits the model two ways on the same data: **weighted** (the population measurement model) and **unweighted** (which, fit with `poLCA`'s assumptions, is what `poLCA` itself targets). Both are aligned to a common label order so their classes correspond. The comparison in Section 11 then shows what the design weights change.

```
# Fit weighted and unweighted at K_star. The weighted fit is the reference label
# order; the unweighted fit is aligned to it so "Class 1" means the same thing in
# every table and figure that compares the two.
fit_w <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K_star, cfg$n_starts)
fit_u <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared)), cats, cfg$items,
                          K_star, cfg$n_starts), fit_w)
```

10 Design-based confidence intervals via the stratified jackknife

10.1 Why the survey weights alone are not enough

A natural question at this point is: the data already came with weights, so why not simply use them and be done? The answer is that survey weights do **one** of the two jobs that honest estimation requires, and the confidence intervals depend on the other.

The weights do the first job, **point estimation**: they make the prevalences and response probabilities describe the *population* rather than the achieved sample. That is precisely the weighted M-step in Section 7, $\hat{\pi}_k = \sum_i w_i p_{ik} / \sum_i w_i$, and the script does exactly that. If the only goal were the estimates themselves, "just use the weights" would be the complete answer.

The weights do **not** do the second job, **variance estimation**, because the precision of a survey estimate depends on *how the sample was selected*, not only on the weights. Two surveys can share an identical weight column yet differ greatly in precision: a sample of 1,200 people drawn as 20 clusters carries far less information than 1,200 people drawn independently, because respondents within a cluster tend to resemble one another, so each additional person in a cluster adds less new information than an independent person would. The weight attached to a respondent says nothing about which cluster they came from; the **design** (the stratum and PSU identifiers) is what carries that information. This is why the code below declares `svydesign(ids = ~psu, strata = ~stratum, weights = ~w)` and builds replicate weights from it, rather than working from the weight column alone.

The practical consequence of ignoring this is concrete and one-directional. If you weighted the data and then computed a standard error as though the weighted sample were a simple random sample of independent people, the estimate would be correct but the standard error would typically be **too small**, often by a factor of 1.5 to 2.5 in clustered samples. That inflation factor is the *design effect*. Using the weights for the estimate while ignoring the design for the variance, the single most common error in survey analysis, produces confidence intervals that are falsely narrow and overstates how much the data actually tell you.

There is a second, independent reason the variance cannot come from the weighted fit alone. The weighted objective $\sum_i w_i \log(\cdot)$ is a *pseudo*-log-likelihood, not a genuine likelihood for any probability model, because fractional and unequally weighted contributions do not correspond to a count of independent observations. The standard maximum-likelihood machinery that would ordinarily hand back standard errors from the curvature of the likelihood (the information matrix) therefore does not apply. The variance has to be estimated by *replication*, refitting the model on systematically perturbed versions of the design, which is what the stratified jackknife does and what the next subsection describes.

In one sentence for a non-technical audience: **the weights tell us who the population is, and the design tells us how sure we are**, and we need both.

10.2 Estimating the design-based variance

Confidence intervals must reflect the sampling design, not pretend the data are a simple random sample. The script uses **replicate weights**: the design is declared, converted to a set of stratified jackknife (JKn) replicates, and the entire weighted EM is re-estimated on each replicate. The spread of the replicate estimates gives the design-based variance,

$$\widehat{\mathrm{Var}}(\hat{\theta}) \;=\; \sum_{r} c_r \left(\hat{\theta}^{(r)} - \hat{\theta}\right)^2,$$

where each JKn replicate r deletes one primary sampling unit and reweights the rest of its stratum, with the scale factor c_r that `survey` supplies. With about 20 PSUs across 9 strata this produces roughly 20 replicates and about 11 degrees of freedom for the variance, which is the true precision of the design regardless of the 1,200 respondents; intervals will be honestly wide.

Two robustness points are built in. The design declaration uses `nest = TRUE` so PSUs nested within strata are handled correctly. And **lonely PSUs** (a stratum with a single PSU, which would break JKn) are handled by setting `options(survey.lonely.psu = "adjust")`, which centers such a stratum's contribution at the grand mean rather than erroring; the script reports whether any were encountered. Each replicate refit is **warm-started at the full-sample solution and aligned to it**, which keeps the 20 refits stable, fast, and free of label switching.

```
options(survey.lonely.psu = "adjust")

# Report PSU structure so the analyst sees the degrees of freedom they actually have.
# Note: dplyr::count() is namespace-qualified because matrixStats (loaded for
# rowLogSumExps) also exports a count(), and it loads after dplyr here.
psu_structure <- dat_prepared |>
  dplyr::distinct(.data[[cfg$strata]], .data[[cfg$psu]]) |>
  dplyr::count(.data[[cfg$strata]], name = "psus_in_stratum")
n_psu_total <- nrow(dplyr::distinct(dat_prepared, .data[[cfg$psu]]))
n_strata    <- n_distinct(dat_prepared[[cfg$strata]])
lonely_any  <- any(psu_structure$psus_in_stratum < 2)

# Declare the design and build stratified jackknife replicate weights.
des <- svydesign(ids = as.formula(paste0("~", cfg$psu)),
                 strata = as.formula(paste0("~", cfg$strata)),
                 weights = as.formula(paste0("~", cfg$weight)),
                 data = dat_prepared, nest = TRUE)
rep_des <- as.svrepdesign(des, type = "JKn")
```

```r
# Flatten a fit's parameters into one named vector, in a fixed order, for variance bookkeepi
param_names <- c(
  paste0("pi[", seq_len(K_star), "]"),
  unlist(imap(cats, function(Cj, j)
    as.vector(outer(seq_len(Cj), seq_len(K_star),
                    function(c, k) sprintf("rho[%s,cat%d,class%d]", cfg$items[j], c, k)))))
)
flatten_fit <- function(fit) set_names(c(fit$pi, unlist(map(fit$rho, as.vector))), param_nam

# The statistic survey::withReplicates evaluates on each replicate: refit the weighted EM
# on the replicate weights, warm-started at the full-sample fit and aligned to it.
theta_fun <- function(w_rep, data) {
  inp <- make_inputs(data, cfg$items, cats)
  fit <- em_run(inp$Y, inp$OH, cats, w_rep, K_star,
                init = fit_w[c("pi", "rho")], maxit = 400L)
  flatten_fit(align_to(fit, fit_w))
}

rep_var <- withReplicates(rep_des, theta_fun)
est_vec <- flatten_fit(fit_w)
se_vec  <- sqrt(diag(vcov(rep_var)))

# Assemble a tidy table of every probability with its design-based 95% CI.
ci_tbl <- tibble(parameter = param_names,
                 estimate = est_vec,
                 se = se_vec,
                 lo = pmax(0, est_vec - 1.96 * se_vec),
                 hi = pmin(1, est_vec + 1.96 * se_vec))
```

The class-size (inclusion) probabilities with their design-based intervals:

```r
ci_tbl |>
  filter(str_starts(parameter, "pi")) |>
  mutate(Class = paste("Class", seq_len(K_star)),
         `Prevalence` = round(estimate, 3),
         `SE` = round(se, 4),
         `95% CI` = sprintf("[%.3f, %.3f]", lo, hi)) |>
  dplyr::select(Class, Prevalence, SE, `95% CI`) |>
  knitr::kable(align = "lrrr",
               caption = "Population class prevalences (inclusion probabilities) with design
```

Table 3: Population class prevalences (inclusion probabilities) with design-based JKn confidence intervals.

Class	Prevalence	SE	95% CI
Class 1	0.561	0.0228	[0.517, 0.606]
Class 2	0.439	0.0228	[0.394, 0.483]

```
rho_ci <- ci_tbl |>
  filter(str_starts(parameter, "rho")) |>
  extract(parameter, c("item", "cat", "class"),
          "rho\\[(.*),cat(\\d+),class(\\d+)\\]", convert = TRUE) |>
  mutate(Class = factor(paste("Class", class)),
         item = factor(item, levels = cfg$items))

ggplot(rho_ci, aes(cat, estimate, color = Class, group = Class)) +
  geom_line(linewidth = 0.4) +
  geom_pointrange(aes(ymin = lo, ymax = hi), size = 0.25, linewidth = 0.4) +
  facet_wrap(~ item, scales = "free_x", ncol = 3) +
  scale_color_manual(values = class_pal(K_star)) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(x = "Response category (highest code = Don't know, if kept)",
       y = "P(response | class)", color = NULL) +
  theme_lca()
```

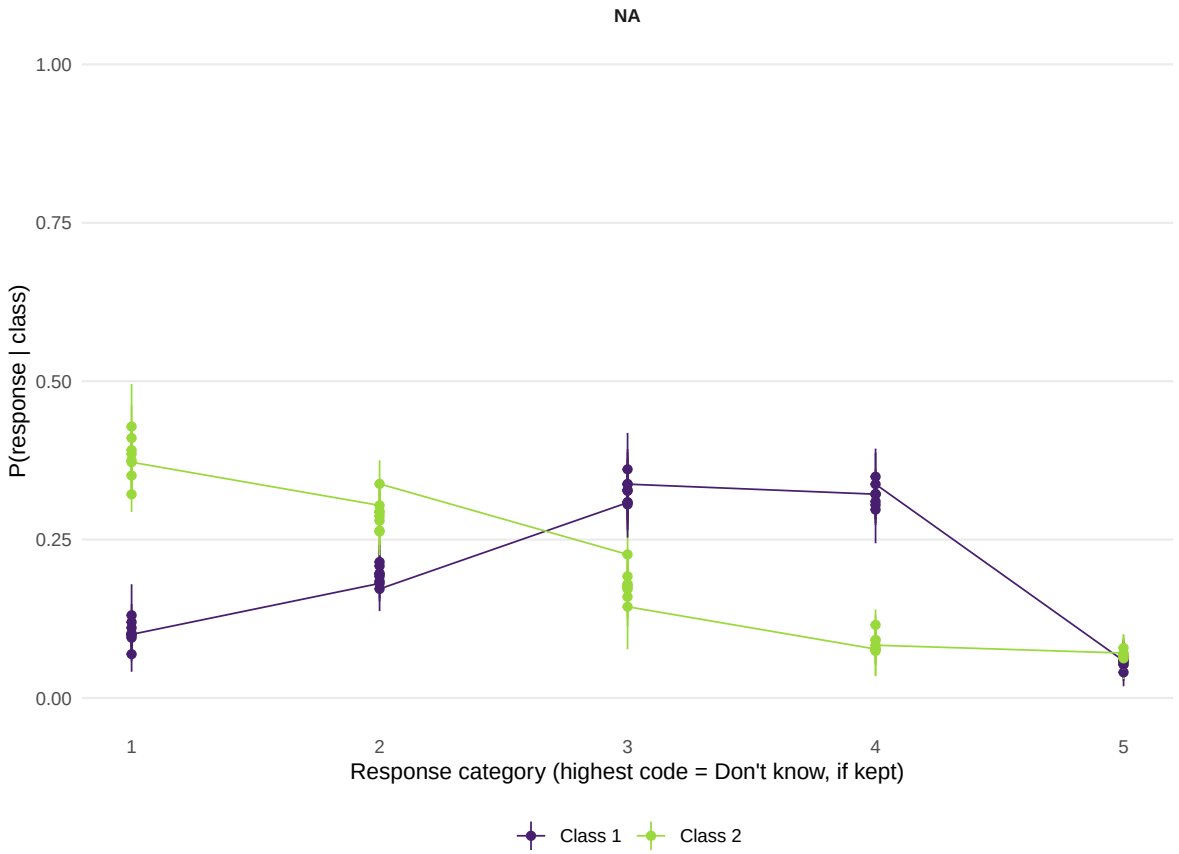

Figure 2: Population item-response (conditional) probabilities with design-based 95% intervals. Each panel is an item; points are classes at each response category. This is the measurement model that defines the respondent types.

11 Benchmark against `poLCA`, and a validation check

Two comparisons with `poLCA` close the loop. First, a **validation**: when all weights are set to one, the weighted EM and `poLCA` maximize the identical likelihood, so their estimates must agree up to label order and convergence tolerance. Confirming this is the strongest evidence that the custom EM is implemented correctly. Second, a **substantive benchmark**: `poLCA` fit normally (unweighted) estimates the achieved-sample structure, and its gap from the weighted estimates measures how much the design matters for this typology. The script also reports the K that `poLCA`'s own BIC would select, since informative sampling can change the apparent number of classes.

```r
# poLCA needs a data frame of items coded 1..C with no NA (it uses na.rm).
polca_df <- dat_prepared |> dplyr::select(all_of(cfg$items)) |> as.data.frame()
polca_f  <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ") ~ 1"))

# --- substantive benchmark: poLCA at K_star, unweighted ---
set.seed(7)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
            maxiter = 5000, na.rm = TRUE, verbose = FALSE)
pl_aligned <- align_to(list(pi = as.numeric(pl$P),
                            rho = map(pl$probs, t),
                            post = pl$posterior), fit_w)

# --- poLCA's own BIC-selected K across the candidate range ---
polca_bic <- map_dfr(cfg$K_range, function(K) {
  set.seed(10 + K)
  m <- poLCA(polca_f, data = polca_df, nclass = K, nrep = 5,
             maxiter = 5000, na.rm = TRUE, verbose = FALSE)
  tibble(K = K, poLCA_BIC = m$bic)
})
polca_K <- polca_bic |> slice_min(poLCA_BIC, n = 1) |> pull(K) |> first()

# --- validation: weighted EM with all weights = 1 should match poLCA ---
fit_unit <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared)), cats, cfg$items,
                             K_star, cfg$n_starts), fit_w)
max_pi_gap  <- max(abs(fit_unit$pi - pl_aligned$pi))
max_rho_gap <- max(map2_dbl(fit_unit$rho, pl_aligned$rho, ~ max(abs(.x - .y))))
```

```r
tibble(`Latent class` = paste("Class", seq_len(K_star)),
       `Population (weighted)` = round(fit_w$pi, 3),
       `Sample (poLCA)` = round(pl_aligned$pi, 3),
       `Difference (pp)` = round((fit_w$pi - pl_aligned$pi) * 100, 1)) |>
  knitr::kable(align = "lrrr",
               caption = "Class prevalence: weighted (population) versus unweighted poLCA (ac
```

Table 4: Class prevalence: weighted (population) versus unweighted poLCA (achieved sample). The difference is the imprint of the design.

Latent class	Population (weighted)	Sample (poLCA)	Difference (pp)
Class 1	0.561	0.598	-3.7
Class 2	0.439	0.402	3.7

The validation check returns a maximum class-size discrepancy of 5×10^{-4} and a maximum response-probability discrepancy of 3×10^{-4} between the unit-weight EM and `poLCA`; values near zero confirm the EM is correct. The weighted BIC selected 2 classes, and `poLCA`'s unweighted BIC selected 2.

```
bind_rows(
  tibble(Class = paste("Class", seq_len(K_star)), prevalence = fit_w$pi,
         Estimator = "Weighted (population)"),
  tibble(Class = paste("Class", seq_len(K_star)), prevalence = pl_aligned$pi,
         Estimator = "Unweighted (poLCA)")
) |>
  ggplot(aes(Class, prevalence, fill = Estimator)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_fill_manual(values = wu_pal) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(x = NULL, y = "Class prevalence", fill = NULL) +
  theme_lca()
```

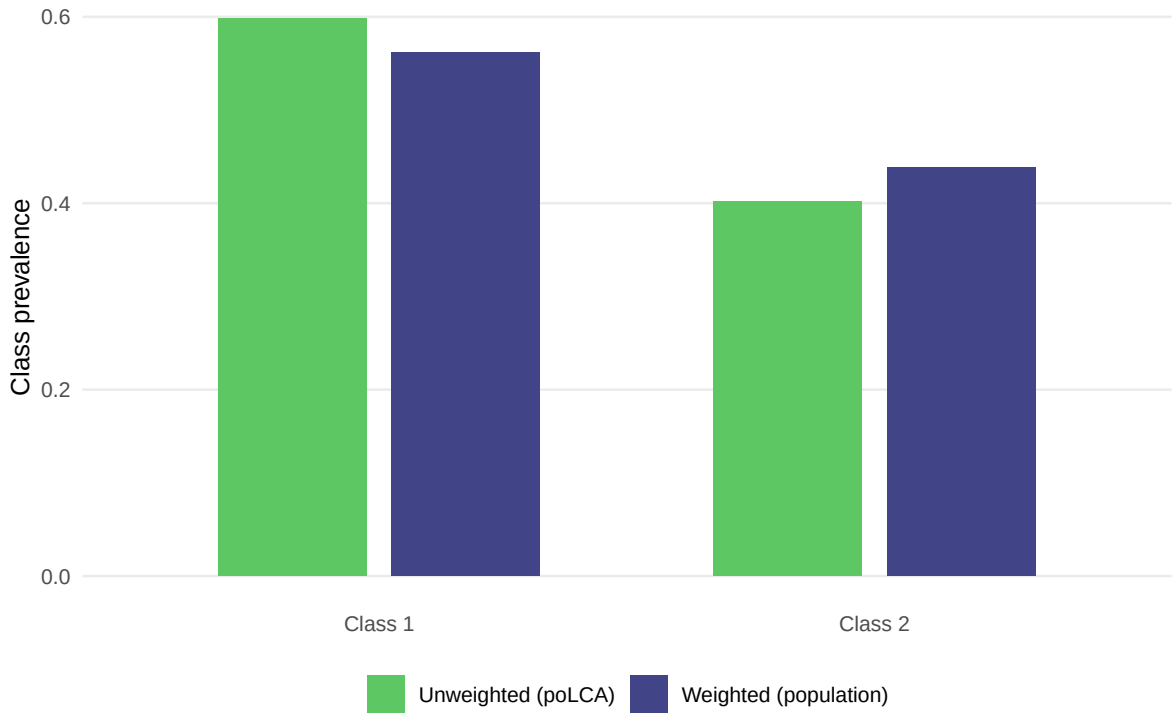

Figure 3: Population (weighted) versus achieved-sample (poLCA) class prevalences. Separation between the two indicates an informative design for this typology.

12 How much the design matters, against known truth

Because the worked example is simulated, the script can show the *gain* from weighting directly: how close each estimator's prevalences are to the population truth. On real data this section instead reports only the weighted-versus-unweighted divergence (Section 11), because the truth is unknown and no error can be computed without fabricating a target.

This comparison is only well defined when the selected number of classes equals the number used to generate the data. When the weighted BIC selects a different number (a central, weakly separated class can be merged under BIC, for instance), a class-by-class prevalence comparison has no meaning, and the script says so rather than forcing a misleading table. When the counts do match, the estimated classes are first aligned to the *true* generating classes by the weighted overlap of their modal assignments, so each row compares like with like.

```
# Valid only when the selected K equals the generating K. Otherwise report that
# plainly: a class-by-class comparison across different K is not defined.
K_match <- (K_star == K_true)

if (K_match) {
  # Align estimated classes to the TRUE classes by the weighted overlap of modal
  # assignment with the known generating class (Hungarian algorithm, maximizing overlap).
  ov_df <- tibble(wt  = dat_prepared[[cfg$weight]],
                  est = max.col(fit_w$post, ties.method = "first"),
                  tru = dat_prepared$true_class)
  overlap <- as.matrix(xtabs(wt ~ est + tru, data = ov_df))
  to_true <- as.integer(clue::solve_LSAP(overlap, maximum = TRUE))  # estimated row -> true c
  est_in_true_order <- integer(K_true)
  est_in_true_order[to_true] <- seq_len(K_star)                     # true col -> estimated r

  truth_tbl <- tibble(
    `Latent class (true)` = paste("Class", seq_len(K_true)),
    `Truth`    = round(pop_pi_true, 3),
    `Weighted` = round(fit_w$pi[est_in_true_order], 3),
    `poLCA`    = round(pl_aligned$pi[est_in_true_order], 3)) |>
    mutate(`|Weighted - Truth|` = abs(Weighted - Truth),
           `|poLCA - Truth|`    = abs(poLCA - Truth))

  err_w <- round(max(truth_tbl$`|Weighted - Truth|`) * 100, 1)
  err_u <- round(max(truth_tbl$`|poLCA - Truth|`) * 100, 1)

  truth_text <- sprintf(
    "the weighted estimator's prevalences fall within %s percentage points of the population
    err_w, err_u)
```

```
  display_tbl <- truth_tbl |>
    mutate(across(c(`|Weighted - Truth|`, `|poLCA - Truth|`), ~ round(.x, 3)))
} else {
  err_w <- err_u <- NA_real_
  truth_text <- sprintf(
    "the weighted BIC selected %d classes while the data were generated with %d, so a class-b
    K_star, K_true)
  display_tbl <- tibble(Note = truth_text)
}

knitr::kable(display_tbl, align = "l",
             caption = "Recovery of the known population prevalences (simulation only).")
```

Table 5: Recovery of the known population prevalences (simulation only).

Note
the weighted BIC selected 2 classes while the data were generated with 3, so a class-by-class prevalence comparison is not defined here (this merging or splitting is itself an informative result)

In this run, the weighted BIC selected 2 classes while the data were generated with 3, so a class-by-class prevalence comparison is not defined here (this merging or splitting is itself an informative result). On your own data, delete this section.

13 Writing class membership back and profiling it

Finally, the script attaches each respondent's posterior class probabilities and their **modal class** (the most probable one) to the data, and profiles membership in a target class on demographics. Per the analysis plan, **no posterior threshold is applied**: every respondent is assigned and kept, so the profiling sample is the full design sample and the weights retain their meaning. The posterior probabilities are written alongside the modal class, so a later analysis can use the full uncertainty if desired.

The profiling model is a **design-based logistic regression** of target-class membership on demographics, fit with `survey::svyglm` and `family = quasibinomial()` so the standard errors are linearization-based and respect the design. Two cautions, stated plainly and not hidden: regressing a *modal* class assignment on covariates **attenuates** the coefficients toward zero, because class membership is estimated with error rather than observed; and this is therefore a

descriptive profile of who tends to fall in the class, not unbiased causal inference. For inferential use, the bias-adjusted three-step methods (the BCH and maximum-likelihood corrections) are the grounded upgrade, and they operate on exactly this design and regression.

```
# Attach posteriors and modal class to the respondent data.
post_w <- fit_w$post
colnames(post_w) <- paste0("post_class", seq_len(K_star))
dat_out <- dat_prepared |>
  bind_cols(as_tibble(post_w)) |>
  mutate(modal_class = max.col(post_w, ties.method = "first"),
         max_posterior = post_w[cbind(seq_len(n()), max.col(post_w, ties.method = "first"))])

# Choose the target class (default: most prevalent).
target <- if (is.na(cfg$target_class)) which.max(fit_w$pi) else cfg$target_class

# Design-based logistic regression of target-class membership on demographics.
prof_des <- svydesign(ids = as.formula(paste0("~", cfg$psu)),
                      strata = as.formula(paste0("~", cfg$strata)),
                      weights = as.formula(paste0("~", cfg$weight)),
                      data = dat_out |> mutate(y_target = as.integer(modal_class == target)),
                      nest = TRUE)
prof_formula <- reformulate(cfg$aux, response = "y_target")
prof_fit <- svyglm(prof_formula, design = prof_des, family = quasibinomial())

prof_tbl <- broom::tidy(prof_fit, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  mutate(`Odds ratio` = exp(estimate),
         `OR 95% CI` = sprintf("[%.2f, %.2f]", exp(conf.low), exp(conf.high)))
```

```
prof_tbl |>
  transmute(Term = term,
            `Log-odds` = round(estimate, 3),
            `SE` = round(std.error, 3),
            `Odds ratio` = round(`Odds ratio`, 2),
            `OR 95% CI`) |>
  knitr::kable(align = "lrrrr",
               caption = paste0("Design-based logistic profile of membership in Class ", targ
                                " (the target class). Coefficients are descriptive and attenu
```

Table 6: Design-based logistic profile of membership in Class 1 (the target class). Coefficients are descriptive and attenuated by classification error.

Term	Log-odds	SE	Odds ratio	OR 95% CI
age_grp35-54	-0.196	0.155	0.82	[0.56, 1.20]
age_grp55+	-0.129	0.167	0.88	[0.58, 1.32]
educationHS or less	0.138	0.157	1.15	[0.78, 1.69]
educationSome college	0.153	0.139	1.17	[0.83, 1.64]
regionInterior	-0.352	0.128	0.70	[0.51, 0.96]

```
prof_tbl |>
  mutate(term = fct_reorder(term, `Odds ratio`)) |>
  ggplot(aes(`Odds ratio`, term)) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey55") +
  geom_pointrange(aes(xmin = exp(conf.low), xmax = exp(conf.high)),
                  color = viridisLite::viridis(1, begin = 0.4), size = 0.4) +
  scale_x_log10() +
  labs(x = "Odds ratio (log scale)", y = NULL) +
  theme_lca()
```

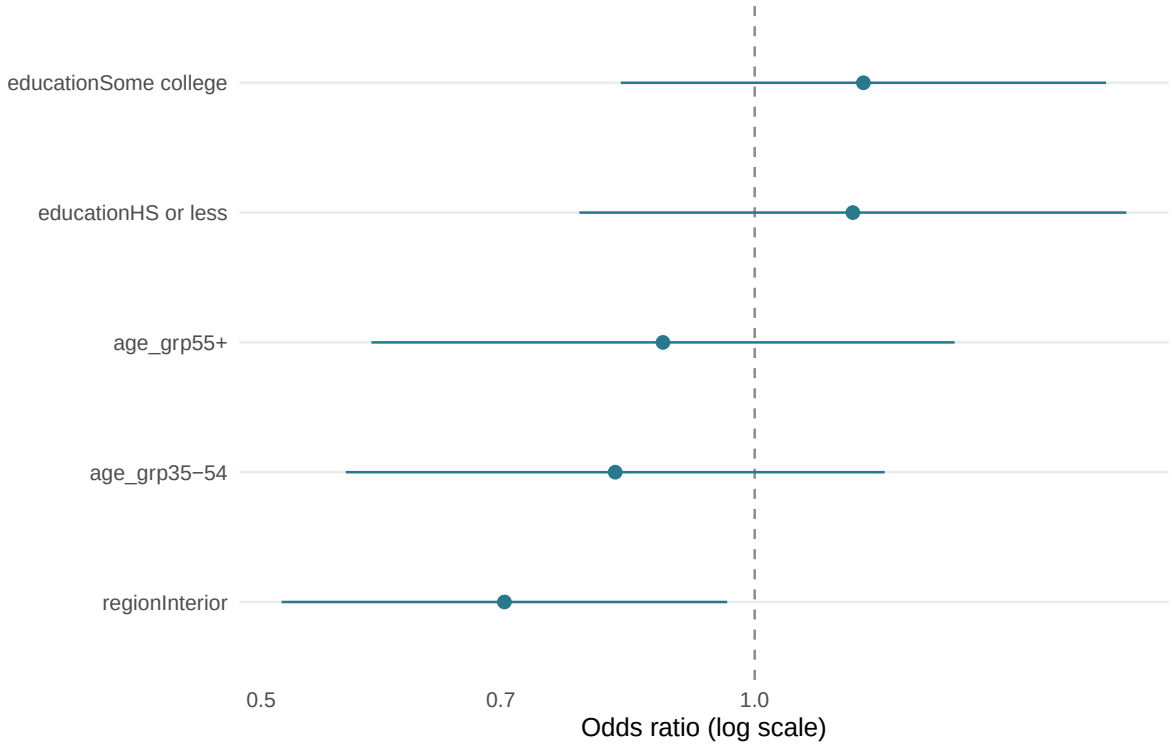

Figure 4: Design-based odds ratios with 95% intervals for membership in the target class. Read as a descriptive profile of who concentrates in the class, not as unbiased effects.

14 Exporting results

```
readr::write_csv(enum, "lca_enumeration.csv")
readr::write_csv(ci_tbl |> mutate(across(where(is.numeric), ~ round(.x, 5))),
                 "lca_measurement_model_with_CIs.csv")
readr::write_csv(dat_out |> dplyr::select(id, all_of(c(cfg$strata, cfg$psu, cfg$weight)),
                                          starts_with("post_class"), modal_class, max_poster
                 "lca_caselevel_assignments.csv")
```

Three files are written: the enumeration table, the full measurement model with design-based confidence intervals, and the case-level assignments (posteriors, modal class, and maximum posterior) ready to merge back into the survey data.

15 Adapting this script to your survey: a checklist

1. **Point `cfg$data` at your data frame** and delete Section 5 (the simulation) and Section 12 (the truth comparison), which exist only because the example is simulated.
2. **Set `cfg$items`** to your indicator columns, coded as consecutive integers from 1. Decide whether **"Don't know" is a category** (`dk_as_category = TRUE`, the default) or **missing** (`FALSE`); the script handles both, and the category-count table after Section 6 confirms which took effect.
3. **Set `cfg$strata`, `cfg$psu`, `cfg$weight`** to your design columns. This is the analogue of `svyset psu [pweight = w], strata(stratum)`. If your file ships **replicate weights** instead of strata and PSU identifiers, replace `svydesign(...)` plus `as.svrepdesign(...)` with a single `svrepdesign(repweights = ..., weights = ..., type = ...)`; the rest of the variance code is unchanged.
4. **Set `cfg$aux`** to the demographics you want in the profile, confirming they are **not** among the indicators (otherwise the profile is circular).
5. **Run, then read enumeration first.** Choose K on the weighted BIC, sanity-checked against entropy and interpretability; the validation check should show the EM matching `poLCA` at unit weights before you trust the weighted output.
6. **Mind the degrees of freedom.** Your design-based intervals carry roughly (number of PSUs minus number of strata) degrees of freedom; with few PSUs they will be wide, and that width is honest.
7. **Treat the profile as descriptive.** If you need inferential statements about demographic predictors, move to a bias-adjusted three-step model, keeping the design-based variance.

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: America/New_York
```

```
tzcode source: internal

attached base packages:
[1] grid      stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
 [1] clue_0.3-68          poLCA_1.6.0.2        MASS_7.3-65
 [4] scatterplot3d_0.3-45 survey_4.5           survival_3.8-6
 [7] Matrix_1.7-5         matrixStats_1.5.0    lubridate_1.9.5
[10] forcats_1.0.1        stringr_1.6.0        dplyr_1.2.1
[13] purrr_1.2.2          readr_2.2.0          tidyr_1.3.2
[16] tibble_3.3.1         ggplot2_4.0.3        tidyverse_2.0.0
[19] gander_0.2.0         ellmer_0.4.1

loaded via a namespace (and not attached):
 [1] gtable_0.3.6       xfun_0.57          httr2_1.2.2        lattice_0.22-9
 [5] tzdb_0.5.0         vctrs_0.7.3        tools_4.6.0        generics_0.1.4
 [9] parallel_4.6.0     cluster_2.1.8.2    pkgconfig_2.0.3    RColorBrewer_1.1-3
[13] S7_0.2.2           lifecycle_1.0.5    compiler_4.6.0     farver_2.1.2
[17] tinytex_0.59       mitools_2.4        htmltools_0.5.9    yaml_2.3.12
[21] crayon_1.5.3       pillar_1.11.1      tidyselect_1.2.1   digest_0.6.39
[25] stringi_1.8.7      labeling_0.4.3     splines_4.6.0      fastmap_1.2.0
[29] cli_3.6.6          magrittr_2.0.5     broom_1.0.13       withr_3.0.2
[33] scales_1.4.0       backports_1.5.1    rappdirs_0.3.4     bit64_4.8.2
[37] timechange_0.4.0   rmarkdown_2.31     bit_4.6.0          otel_0.2.0
[41] hms_1.1.4          evaluate_1.0.5     knitr_1.51         viridisLite_0.4.3
[45] rlang_1.2.0        Rcpp_1.1.1-1.1     glue_1.8.1         DBI_1.3.0
[49] coro_1.1.0         rstudioapi_0.18.0  vroom_1.7.1        jsonlite_2.0.0
[53] R6_2.6.1
```